

Bab: Branching dan Control Statement di R

Pertemuan 4

Disusun oleh : Weksi Budiaji & Ferdian Bangkit Wijaya

Sekuensial / Kaku



Adaptif /
Dinamis



Mengatasi Keterbatasan Program Linear

- Dalam dunia pemrograman, program tidak selalu berjalan lurus dari awal hingga akhir secara sekuensial.
- Control statement atau branching digunakan untuk mengatur jalannya alur eksekusi program.
- Memungkinkan penyusunan program yang lebih fleksibel, dinamis, dan mampu menyesuaikan diri dengan berbagai situasi dunia nyata (seperti menyeleksi data, membuat keputusan perhitungan, atau mengolah data dalam jumlah besar). Tanpa ini, program hanya menjadi sekumpulan perintah linear yang kaku.

6 Kemampuan Utama Control Statement



Konsep Dasar Branching (Percabangan)

Konsep:

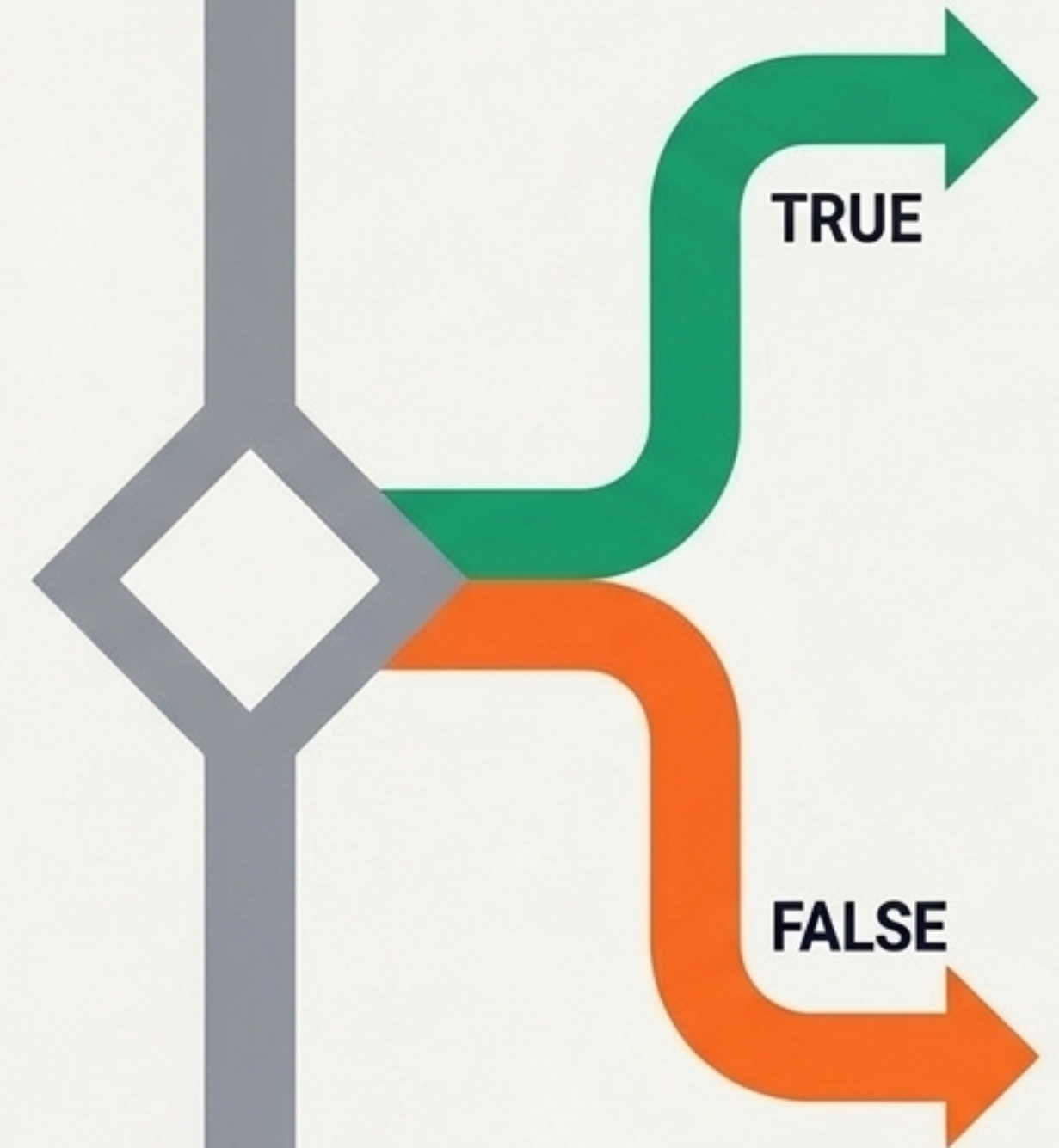
Branching memberikan kemampuan kepada program untuk bertindak secara berbeda tergantung pada kondisi yang terjadi (mengambil keputusan / memilih jalur eksekusi).

Mekanisme:

Jika kondisi benar (TRUE), program mengeksekusi instruksi tertentu. Jika salah (FALSE), program mengeksekusi instruksi lain atau tidak melakukan apa-apa.

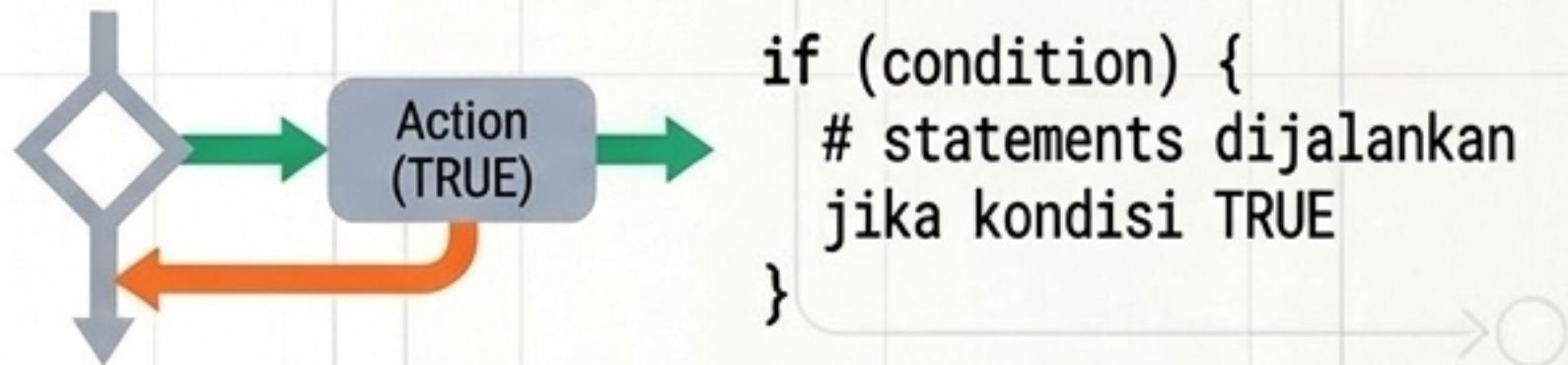
Contoh Penggunaan: Memeriksa bilangan genap/ganjil, menentukan status kelulusan, atau memilih opsi menu aplikasi.

Implementasi R: `if` `if-else` `ifelse` `switch`



Evaluasi Kondisi Tunggal: if

Jika TRUE, blok perintah dijalankan.
Jika FALSE, diabaikan.



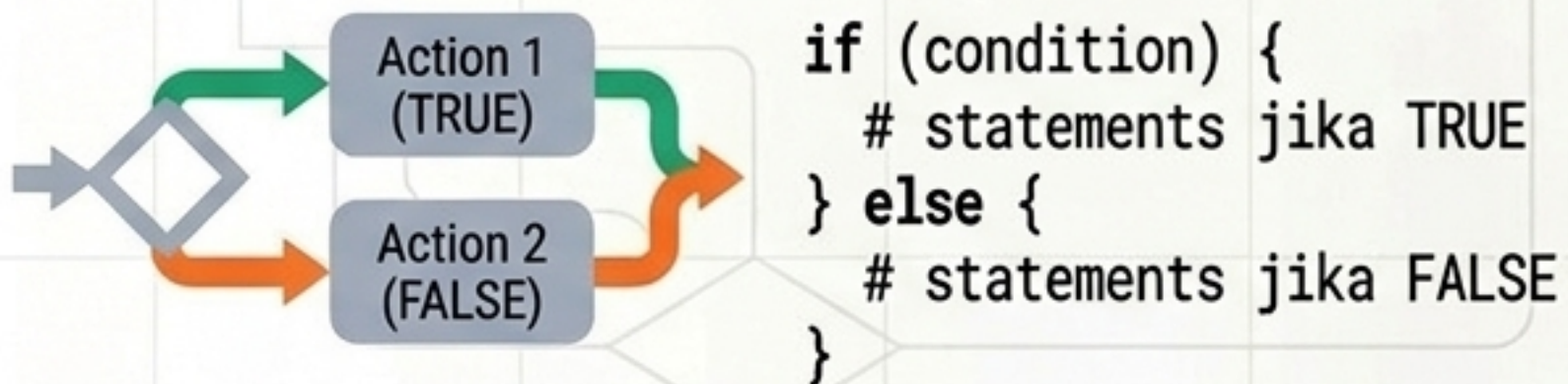
```
x <- 100  
if (x > 10) {  
  print(paste(x, "is greater than 10"))  
}
```

OUTPUT

```
[1] "100 is greater than 10"
```

Dua Jalur Alternatif: if-else

Keputusan biner. Selalu mengeksekusi salah satu blok bergantung pada evaluasi kondisi.



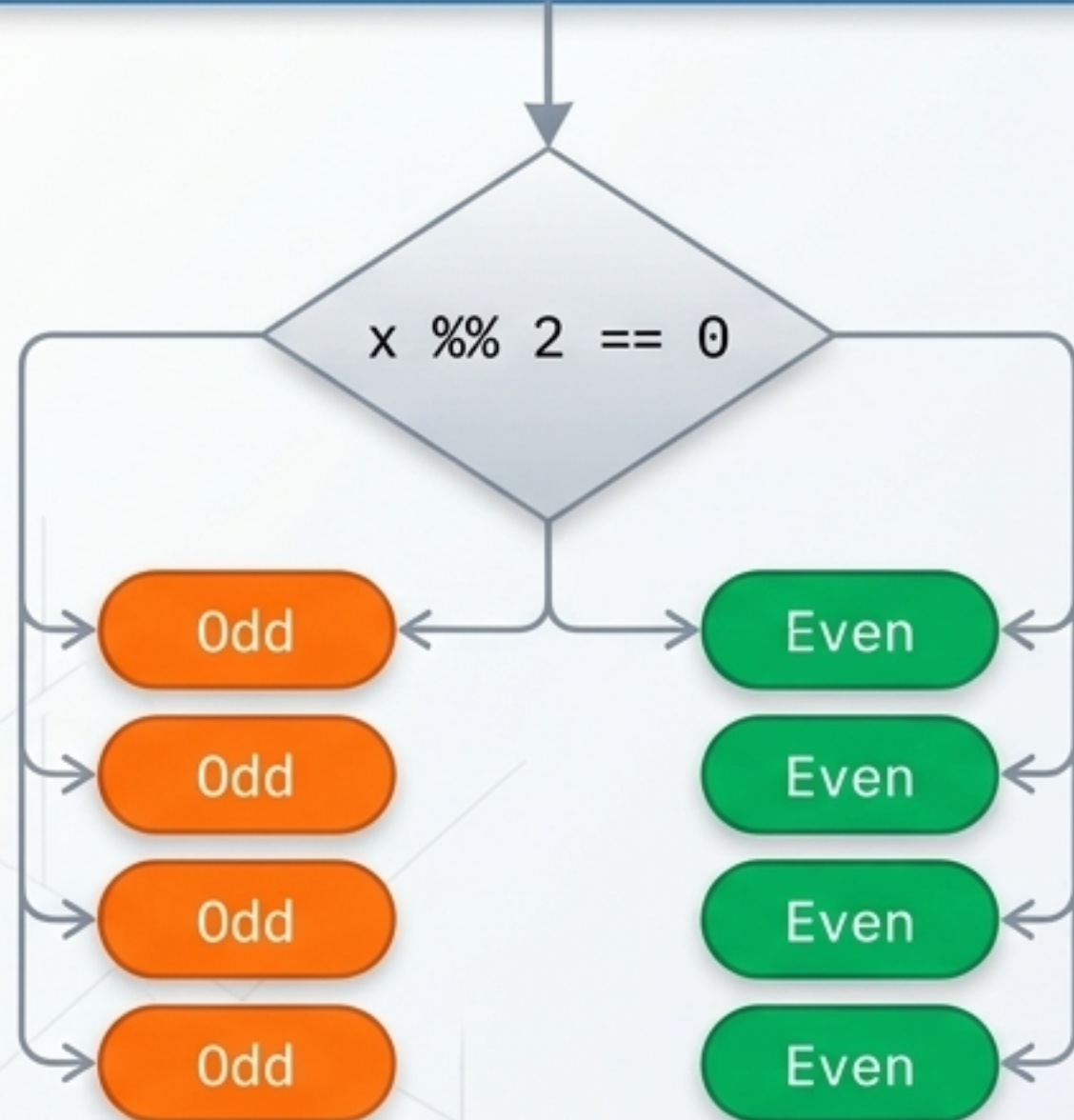
```
x <- 5  
if (x > 10) {  
  print(paste(x, "is greater than 10"))  
} else {  
  print(paste(x, "is less than or equal to 10"))  
}
```

OUTPUT

```
[1] "5 is less than or equal to 10"
```

Efisiensi Data: ifelse (Fungsi Vektorized)

[...] [1, 2, 3, 4, ... 10]



Bentuk **vektORIZED** dari percabangan.

Memeriksa kondisi pada *setiap elemen* vektor sekaligus (lebih efisien daripada loop).

Memiliki 3 argumen: kondisi, nilai jika TRUE, nilai jika FALSE.

`ifelse(condition, value_if_true, value_if_false)`

CODE

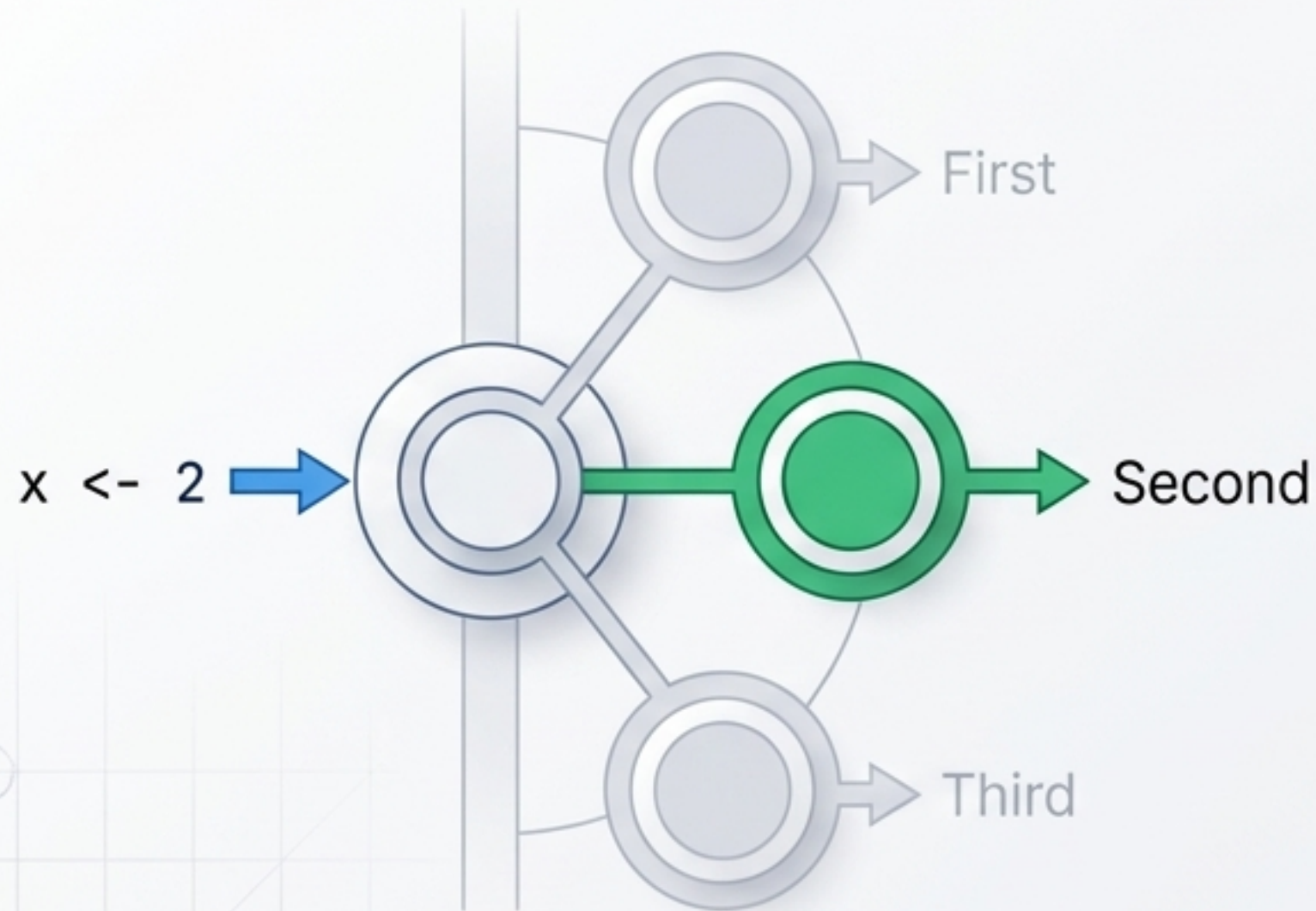
```
x <- 1:10  
ifelse(x %% 2 == 0, "Even", "Odd")
```

OUTPUT

```
[1] "Odd" "Even" "Odd" "Even" "Odd" "Even"  
"Odd" "Even" "Odd" "Even"
```

Penanganan Multi-Kasus: `switch`

Digunakan ketika memiliki lebih dari dua pilihan. Mengeksekusi satu nilai berdasarkan indeks atau nama kasus. Lebih ringkas daripada banyak if-else berurutan (cocok untuk pembuatan menu atau metode perhitungan).



```
switch(index_or_name,  
       case1 = value1,  
       case2 = value2,  
       case3 = value3,  
       ... )
```

```
x <- 2  
result <- switch(x, "First", "Second", "Third")  
print(result)
```

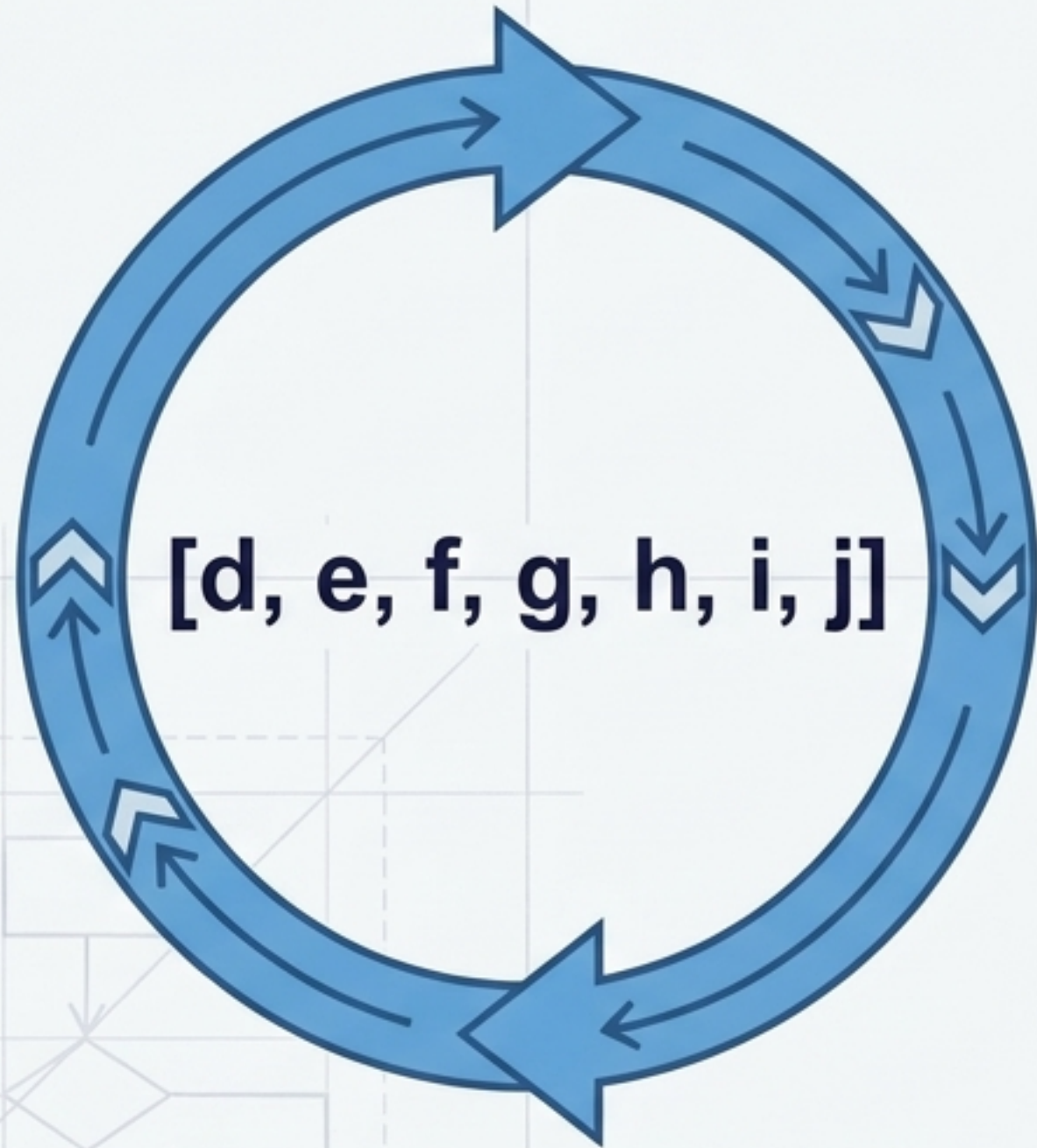
OUTPUT

```
[1] "Second"
```

Sintesis: Matriks Pemilihan Percabangan

Fungsi	Jumlah Kondisi	Sifat	Kasus Penggunaan Terbaik
<code>if</code>	Tunggal	Sekuensial / Skalar	Evaluasi satu syarat sederhana.
<code>if-else</code>	Biner (Dua jalur)	Sekuensial / Skalar	Keputusan biner (misal: lulus/tidak lulus).
<code>ifelse</code>	Biner pada banyak data	Vektorized	Manipulasi vektor / kolom pada data frame.
<code>switch</code>	Multi-opsi (>2)	Seleksi via Indeks>Nama	Pembuatan menu, seleksi metode tunggal yang ringkas.

Pengantar Looping & Iterasi Terstruktur (for)



Melakukan perulangan berurutan pada setiap elemen dalam vektor, list, atau objek serupa. Bermanfaat untuk memproses data frame baris demi baris atau mencetak isi vektor.

```
for (value in vector) {  
  # statements yang akan dijalankan  
}
```

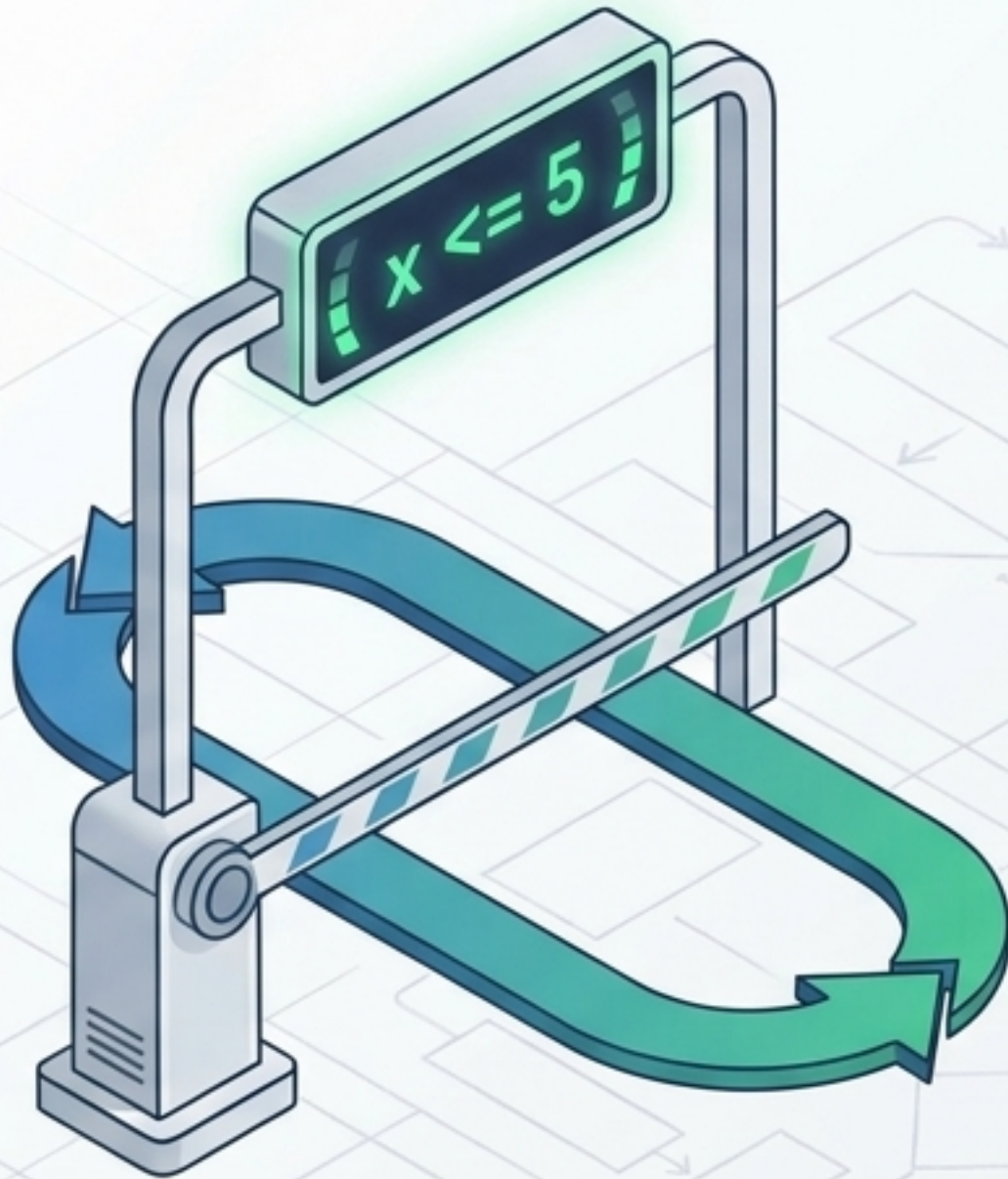
CODE

```
x <- letters[4:10]  
for (i in x) {  
  print(i)  
}
```

OUTPUT

```
[1] "d"    [1] "f"    [1] "h"    [1] "j"  
[1] "e"    [1] "g"    [1] "i"
```

Iterasi Bersyarat (while)



Mengulang perintah *selama* kondisi masih bernilai TRUE. Berbeda dengan for, jumlah iterasi tidak diketahui di awal, melainkan ditentukan oleh evaluasi kondisi secara dinamis.

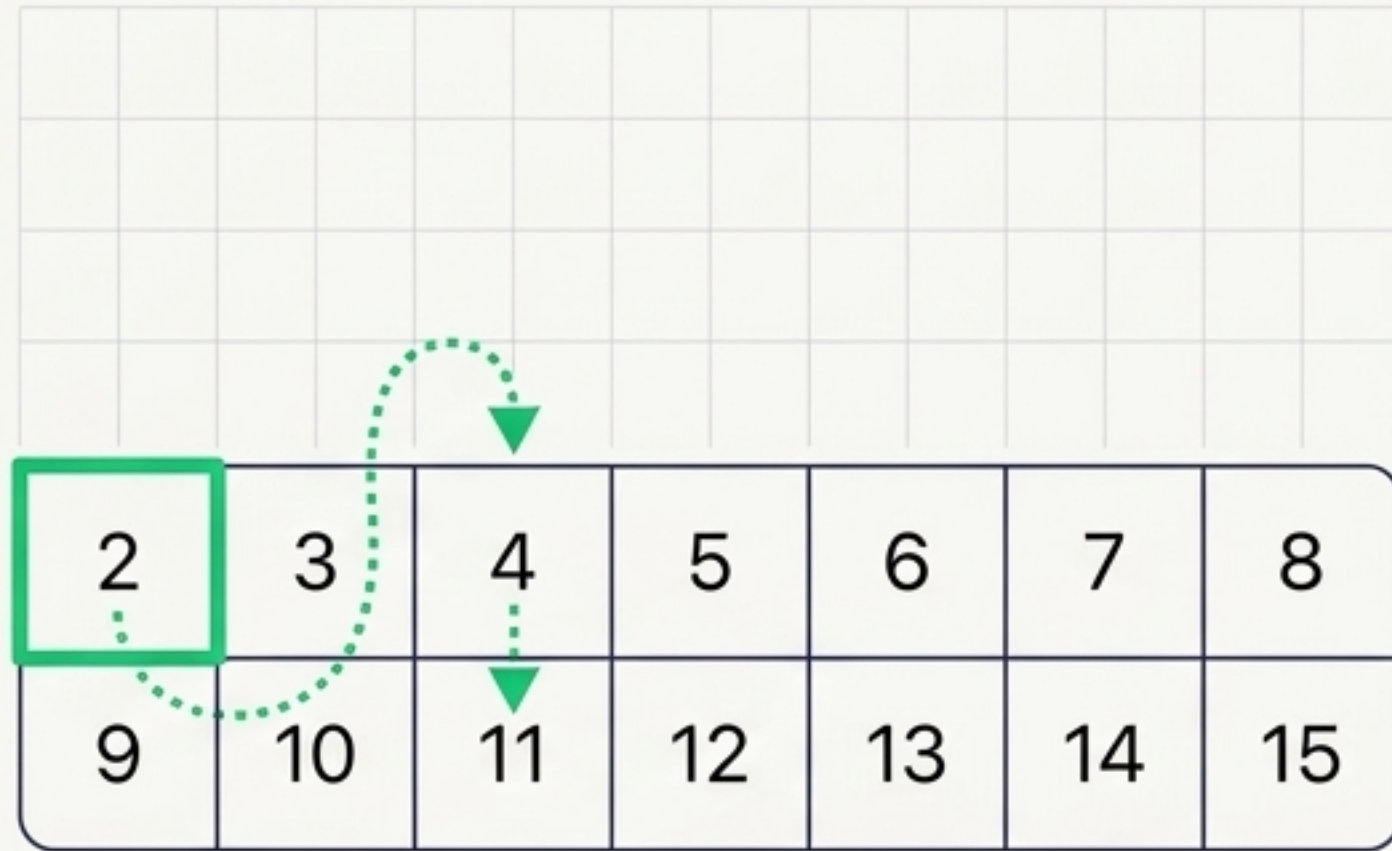
```
while (condition) {  
  # statements dijalankan berulang  
}
```

CODE

```
x <- 1  
while (x <= 5) {  
  print(x)  
  x <- x + 1  
}
```

OUTPUT [1] 1 [1] 2 [1] 3 [1] 4 [1] 5

Eksekusi Multi-Dimensi (Nested Loops)



2	3	4	5	6	7	8
9	10	11	12	13	14	15

Loop berada di dalam loop lain. Digunakan untuk mengakses data dua dimensi (matriks/tabel), melakukan perulangan pada baris sekaligus kolom.

CODE

```
# Membuat matriks
m <- matrix(2:15, 2)

# Loop pada baris dan kolom
for (r in seq(nrow(m))) {
  for (c in seq(ncol(m))) {
    print(m[r, c])
  }
}
```

OUTPUT

```
[1] 2 [1] 4 [1] 6 [1] 8 [1] 10 [1] 12 [1] 14
[1] 3 [1] 5 [1] 7 [1] 9 [1] 11 [1] 13 [1] 15
```

Loop Tak Terbatas & Penghentian (repeat & break)



`repeat` membuat perulangan tak terbatas (tidak ada syarat berhenti bawaan). Membutuhkan perintah `break` untuk keluar dari loop secara manual saat kondisi tertentu terpenuhi agar tidak berjalan selamanya.

```
repeat {  
  # statements; if (condition) { break }  
}
```

CODE

```
x <- 1  
repeat {  
  print(x)  
  x <- x + 1  
  if (x > 5) { break }  
}
```

OUTPUT

```
[1] 1 [1] 2 [1] 3 [1] 4 [1] 5
```

Melewati Iterasi & Mengembalikan Nilai (**next** & **return**)



Melewati iterasi saat ini dan langsung melanjutkan ke iterasi berikutnya tanpa menghentikan seluruh loop.

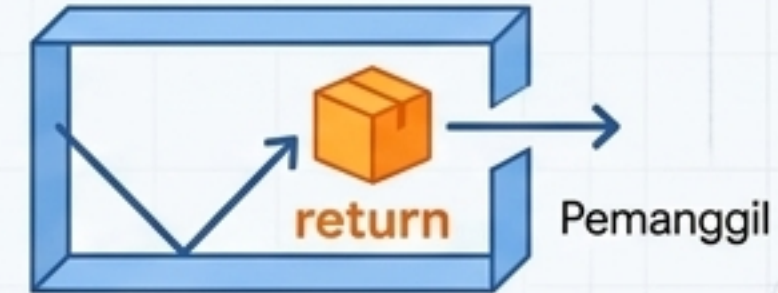
next

CODE

```
for (i in 1:10) {  
  if (i %% 2 != 0) { next } # lewati ganjil  
  print(i)  
}
```

OUTPUT

```
[1] 2 [1] 4 [1] 6 [1] 8 [1] 10
```



Mengembalikan nilai ke pemanggil fungsi dan menghentikan eksekusi fungsi pada titik tersebut.

return(expression)

CODE

```
check_sign <- function(x) {  
  if (x > 0) { return("Positive") }  
  else if (x < 0) { return("Negative") }  
  else { return("Zero") }  
}
```

```
check_sign(-3)
```

OUTPUT

```
[1] "Negative"
```

Sintesis: Matriks Kontrol Alur Eksekusi

Fungsi / Modifier	Jumlah Iterasi / Aksi	Syarat Berhenti / Sifat	Penggunaan Utama
<code>for</code>	Diketahui (Sesuai panjang vektor)	Elemen terakhir tercapai	Iterasi data terstruktur.
<code>while</code>	Tidak diketahui	Kondisi bernilai FALSE	Iterasi berbasis evaluasi kondisi.
<code>repeat</code>	Tak terbatas	Harus via `break`	Loop permanen hingga interupsi manual.
<code>break</code>	Menghentikan loop	Keluar total	Syarat interupsi kritis.
<code>next</code>	Melewati iterasi	Lanjut ke step berikutnya	Mengabaikan kasus spesifik.
<code>return</code>	Mengakhiri fungsi	Mengirim nilai balik	Ekspor hasil perhitungan.

Tugas Praktikum (Evaluasi Pembelajaran)

CARD 1

Tugas 1

Diberikan sebuah vektor numerik, cari mediannya dengan fungsi yang dibuat sendiri (bukan fungsi dari R).

CODE

```
# Contoh penggunaan  
x <- c(3, 7, 2, 9, 5)  
find_median(x)
```

CARD 2

Tugas 2

Diberikan vektor numerik p=4 (3, 4, 5, 6). Hitung mean, tambahkan nilai mean ke vektor, hitung lagi. Ulangi sampai panjang vektor = 10. Implementasikan dalam fungsi R.

CODE

```
# Contoh penggunaan  
x <- c(3, 4, 5, 6)  
mean_extend(x)
```

CARD 3

Tugas 3

Diberikan sebuah vektor numerik, buat fungsi untuk menghitung jumlah bilangan genap di dalam vektor.

CODE

```
# Contoh penggunaan  
x <- c(3, 4, 7, 10, 12, 15)  
hitung_genap(x) # hasilnya 3
```

CARD 4

Tugas 4

Buat fungsi yang menerima sebuah matriks numerik, lalu hitung jumlah semua elemen genap di dalam matriks.

CODE

```
# Contoh penggunaan  
m <- matrix(1:12, nrow = 3, ncol = 4)  
m  
hitung_genap_matriks(m)
```